

TD 9 - Dynamic Programming

Notation and reminders : We denote by $\Pi(X)$ the set of probability distributions over a space X .

A MDP is defined by a state space $\mathcal{S}$, an action space $\mathcal{A}$, a transition function $T : \mathcal{S} \times \mathcal{A} \rightarrow \Pi(\mathcal{S})$ and a reward function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$. For convenience, we write $p(s'|a, s) = T(s, a, s')$ for the probability of reaching s' from s by taking action a . A *discount factor* γ and/or an initial state distribution ρ_0 are sometimes added.

A stochastic policy $\pi : \mathcal{S} \rightarrow \Pi(\mathcal{A})$ governs the behavior of an agent and assigns to that agent a probability $\pi(a|s)$ of choosing action a in every state s .

An agent equipped with a policy π interacting with a MDP representing its environment produces trajectories $\tau^\pi = \{s_0, a_0, r_1, s_1, a_2, r_2, \dots\}$ with $r_{t+1} = r(s_t, a_t, s_{t+1})$ ¹. These trajectories may be infinite or bounded by a horizon H . We denote by $R(\tau)$ the return of a trajectory τ , computed in the discounted case as $R(\tau) = \sum_{t \geq 0} \gamma^t r_{t+1}$.

The value function of state s under policy π is defined by $V^\pi(s) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s]$.

Similarly, the action-value function $Q^\pi(s, a) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s, a_0 = a]$ defines the value of being in state s , taking action a , and then following policy π .

We also define the optimal value function $V^*(s) = \max_\pi V^\pi(s)$ and the optimal action-value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$.

The general objective of this problem set is to show that the *value iteration* and *policy iteration* algorithms converge to optimal value functions and action-value functions.

Exercice 1 – Bellman Operator, Policy Evaluation and Policy Iteration

Q 1.1 What are the relationships between Q^π and V^π , when the policy π is respectively deterministic and stochastic?

Q 1.2 In the stochastic case, show that

$$\begin{aligned} V^\pi(s) &= \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma V^\pi(s')] \\ &= \mathbb{E}_{\substack{a \sim \pi \\ s_{t+1} \sim p(\cdot|s_t, a_t)}} [r(s, a, s') + \gamma V^\pi(s')]. \end{aligned}$$

Q 1.3 We call the Bellman operator T^π the operator

$$V_{k+1}(s) = (T^\pi V_k)(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma V_k(s')].$$

Show the monotonicity property of the operator : if $\forall s, V_1(s) \leq V_2(s)$, then $\forall s : T^\pi V_1(s) \leq T^\pi V_2(s)$.

Q 1.4 Show that the repeated application of the Bellman operator to a value function V causes it to converge to a fixed point.

Q 1.5 The *policy iteration* algorithm works, at each iteration k , in two steps :

1. Evaluate V^{π_k} until convergence under the current stationary policy π_k .
2. Update the policy π_{k+1} according to a greedy strategy :

$$\forall s, \pi_{k+1}(s) = \operatorname{argmax}_a \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma V^{\pi_k}(s')]$$

Deduce the convergence of *policy iteration* from the monotonicity property proved in the previous question.

Exercice 2 – Bellman Optimality Operator and Value Iteration

¹. We more commonly use $r_{t+1} = r(s_t, a_t)$ so that the reward does not depend on a stochastic transition. We also sometimes use $r_{t+1} = r(s_t)$ when this modeling better fits the nature of the problem to be solved.

Q 2.1 Show the Bellman optimality equation :

$$V^*(s) = \max_{\pi} V^{\pi}(s) = \max_a \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma V^*(s')].$$

Q 2.2 We call the Bellman optimality operator T^* the operator :

$$\forall s, (T^* V)(s) = \max_a \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma V(s')].$$

Let V_1 and V_2 be two value functions defined for every state s . Show that

$$\|T^*V_1 - T^*V_2\|_{\infty} \leq \gamma \|V_1 - V_2\|_{\infty}.$$

Q 2.3 The *value iteration* algorithm applies, over a given number of iterations, the Bellman optimality operator to update state values : $V^{(k+1)} = T^*V^{(k)}$, where $V^{(k)}$ denotes the value function at iteration k . Deduce the convergence of this algorithm from the contraction property proved in the previous question (assuming the fixed-point property $T^*V^* = V^*$).